

NEXLIB: Aggregation Without Access

Multi-tenant, offline-first library management for government degree colleges on a zero-cost serverless architecture, and what three real defects at 300-institution scale reveal about it

Prepared from the design, implementation, and test history of the NEXLIB system
built for the Higher Education Department, Khyber Pakhtunkhwa, Pakistan

Companion document to Field Notes from a Provincial System, the project's engineering casebook

ABSTRACT

NEXLIB is a library management system deployed across four independent clients — Android, a Python desktop application, a web onboarding portal, and a directorate oversight console — sharing one multi-tenant Firestore backend, built and operated on a free-tier ("Spark") serverless plan with no server-side compute available for authorization or aggregation. This paper documents the system's central architectural device, which we term *client-published aggregation*: each tenant institution periodically writes a small, PII-free rollup of its own counts to a shared registry, and a provincial directorate reads that registry under security rules that grant it visibility without ever granting it access to a single tenant record. We describe the resulting trust model, the governance layer built on top of it, and the two-part verification strategy used to validate a system whose correctness lives almost entirely in declarative security rules rather than server code: 75 behavioral rules assertions against a local emulator, and a synthetic 300-institution scale harness that drives the real application through every write path against deliberately irregular data. The scale harness surfaced three defects invisible to the rules suite and to a passing type-check and build — two of them different symptoms of the same unenforced identity assumption. We report those defects, their fixes, and the generalizable lesson: an alert that detects a data-quality violation does not, on its own, defend the rest of a codebase from that violation's consequences.

Keywords — multi-tenancy, offline-first synchronization, Firestore security rules, serverless architecture, ICT for development, access-control bootstrapping, scale testing

1 Introduction

Government Degree Colleges (GDCs) under Khyber Pakhtunkhwa's Higher Education Department run their libraries the way most under-resourced public institutions do: paper registers, no cross-institution visibility, and no realistic path to a conventional IT department. NEXLIB was built to digitize a single college's library operations first, and only then to let a provincial directorate see the network of colleges as a whole – deliberately in that order, because a system that demands network-wide infrastructure before it is useful to a single librarian never survives contact with the first college that tries it.

That ordering produces the paper's central engineering problem. A directorate needs aggregate visibility – how many books, how many active loans, which colleges have gone quiet – across every institution in the network. Firestore has no native cross-tenant aggregate query, and the project's budget rules out Cloud Functions, which require the pay-as-you-go ("Blaze") plan rather than the free "Spark" plan this deployment runs on. There is, in other words, no trusted server anywhere in this architecture that could compute a network-wide total on the directorate's behalf. Every authorization decision, and every aggregation, has to be expressible as a static, declarative Firestore security rule, evaluated per-document, with no procedural code behind it at all.

This paper documents the pattern that resulted – **client-published aggregation** – the governance layer built on it, and, in its second half, a scale-testing methodology that treated "the build compiles and the type-checker is silent" as necessary but not sufficient evidence of correctness. Run against 300 synthetic institutions seeded with deliberately irregular data, that methodology found three real defects that no unit test, rules test, or manual review at small scale had caught. We report them not as a postmortem but as the paper's main empirical contribution: evidence for what a uniqueness assumption that is *detected* but not *enforced* actually costs a codebase.

Contributions.

- A description of the client-published aggregation pattern as a general technique for read-side cross-tenant visibility on a serverless, no-compute backend, and the trust model it implies.
- A bootstrap-safe access-tier design for the oversight layer built on that pattern, in which the absence of an enrollment record resolves to the most privileged tier rather than the least – the opposite of conventional least-privilege defaults, adopted specifically to make a permission-model migration incapable of locking out an existing administrator.
- A two-layer verification methodology for a system whose authorization logic lives entirely in declarative rules: behavioral rules testing against an emulator, and a synthetic-scale harness that drives the real client UI against realistic, irregular data rather than uniform test fixtures.
- Three concrete defects recovered by that harness at n=300, with root-cause analysis showing two of the three trace to one unenforced assumption surfacing in unrelated parts of the codebase.

2 Related Work

NEXLIB's synchronization model sits inside what Kleppmann et al. call *local-first software*^[1]: each client keeps a usable local copy of its own tenant's data and reconciles with the network opportunistically, rather than treating connectivity as a precondition for the application being usable at all. The tension that framing predicts – between availability during partition and consistency once partition ends – is the applied form of the CAP theorem^[2]: a college's desktop or Android client must remain fully operational while offline, which means the system accepts eventual rather than immediate consistency of the directorate's view of the network, precisely the trade CAP says a partition-tolerant system must make.

On the authorization side, NEXLIB departs from the conventional trusted-server model in which an application server evaluates access-control decisions in procedural code. Firestore Security Rules are a declarative alternative – access control expressed as predicates evaluated by the database itself against the requesting identity and the document in question, with no application server in the loop at all^[3]. This removes an entire class of server-side vulnerability by removing the server, but it also removes the conventional place most testing methodology assumes authorization logic lives, which motivates the behavioral rules-testing approach in Section 4.

The broader context – an information system built for a public institution in a resource-constrained setting – sits inside a long-documented literature on the gap between what such systems are designed to do and what they survive doing once deployed. Heeks's design–reality gap framework^[4] argues that ICT4D project failure is best predicted by the distance between the assumptions a system's design encodes and the operating reality of the institution that must run it; the desktop client's Admin-SDK trust bypass discussed in Section 3.3, adopted specifically because the deployment environment could not be assumed to have a technician available to manage service keys correctly, is exactly this kind of designed-in compromise, and Section 6 returns to it as an open risk rather than a closed one.

3 System Architecture

NEXLIB comprises four independent client applications against one shared Firestore backend: an Android application (Kotlin, layered into repository and use-case modules), a desktop application for library staff (Python, PyQt6), a web onboarding and management portal (Next.js, React), and a directorate oversight console (Next.js, described in Section 5). All four read and write the same tenant schema; only the first three are tenant-facing, and only the fourth exists to look across tenants.

3.1 Tenant schema and isolation

Each college is a tenant rooted at `/institutions/{institutionId}`, with its books, members, issued loans and reservations as subcollections beneath it. Firestore Security Rules enforce that a signed-in account may read or write only the subtree of the institution its own `/users/{uid}` profile names, evaluated per document on every request. This is the isolation boundary the rest of the architecture is built to respect without ever needing to re-express it: nothing above the tenant layer is permitted to read tenant data directly, including the directorate.

3.2 Client-published aggregation

A provincial directorate needs exactly one thing tenant isolation is designed to prevent: a view across every tenant at once. Firestore cannot execute a cross-collection aggregate query, and Cloud Functions – the conventional place to compute one server-side – require a paid plan this deployment does not use. The system resolves this by having each tenant *publish* a small, non-identifying rollup of its own counts to a shared registry, `/directorate_index/{institutionId}`: book, member and loan counts, a last-sync timestamp, and nothing that identifies a patron or a book title. The directorate reads this registry collection directly; no tenant record is ever touched.

DESIGN NOTE

Admission to the network is a separate write path, `/directorate_approvals/{institutionId}`, keyed the same way but owned exclusively by the directorate. A tenant can publish its own rollup, but it cannot admit itself: approval and self-report are different documents under different write rules specifically so that publishing counts and being counted are not the same permission.

Because the aggregation is computed by the tenant about itself, the trust model has a bounded blast radius: a misbehaving or compromised tenant can misstate its own counts, but the security rules make it structurally incapable of writing to another tenant's rollup or forging its own approval. The directorate's visibility is therefore honest by construction with respect to *which* institution a number came from, even though it cannot independently verify that the number is accurate – a limitation the compliance and alerting layer in Section 5 is built to surface rather than hide.

3.3 An asymmetric trust boundary

ARCHITECTURAL LIMITATION, NOT A DEFECT

The desktop client authenticates to Firestore through the Firebase Admin SDK rather than the client SDK, which bypasses security rules entirely by design – a deliberate choice made because the deployment environment cannot be assumed to have a technician on-site able to manage per-user credentials correctly, and an offline-capable desktop tool needs a working local database even when no rule server is reachable. The practical consequence is that a security-rules regression is invisible from the desktop client and fatal only on Android and the web portal, which is why the rules-testing methodology in Section 4 tests behaviorally against the rules themselves rather than against any one client's behavior.

4 Verification Methodology

A system whose authorization logic is entirely declarative has no procedural code path for a conventional unit test to exercise. NEXLIB's verification strategy is built in two layers that test different failure classes, summarized in Table 1.

LAYER	WHAT IT TESTS	SCALE	WHAT IT CANNOT CATCH
Type-check & build	Internal consistency of the code	n/a	Whether the code does the right thing at all
Behavioral rules tests	Every account class × every collection, ALLOW and DENY both asserted	Fixed, small fixture (≈10 accounts)	Emergent behavior only visible under realistic data distributions
Synthetic scale harness	The real client UI, driven end-to-end, against irregular data	300 synthetic institutions	Production network conditions, concurrent writes, device heterogeneity

Table 1. The three verification layers used in this project, and the boundary of each.

4.1 Behavioral rules testing

Using `@firebase/rules-unit-testing` against a local Firestore emulator, the suite instantiates one authenticated context per account class – a tenant's own staff, a tenant's owner, a per-college director role (a tenant-level title, distinct from the network-wide directorate), an outsider tenant, an anonymous caller, and directorate accounts at each of three access tiers – and asserts, for every collection the rules mention, both that the access each class should have succeeds and that everything else fails. At the time of writing this comprises 75 assertions across the tenant schema and eight directorate-only collections, and it is run before every rules deployment specifically because a regression here is, per Section 3.3, invisible from the desktop client and would otherwise ship undetected.

4.2 Synthetic scale harness

Passing rules tests and a clean build establish that the system is internally consistent and that its access boundary holds against a small, clean fixture. Neither establishes that its features behave correctly once real data – irregular, partial, occasionally contradictory – reaches the scale a real network will eventually produce. The scale harness closes that gap by seeding a local emulator with 300 synthetic institutions and then driving the actual directorate console through every screen and every write path with browser automation, rather than inspecting the seeded data directly.

The seed is deliberately not uniform. Edge cases are injected at frequencies chosen to resemble what a real network eventually produces rather than a worst case: roughly two percent of institutions never publish a rollup at all, a further slice publish one that has since gone stale, some report zero books despite an active sync, some carry no recorded contact information, some show an implausibly high proportion of overdue loans, and – the case that turned out to matter most – one institution is represented by two different registry documents, simulating a stale client that republished under a new document identity after a local reset while still claiming the original institution's identity inside its data. This last case exists because the system's own alerting logic already anticipates it: an alert exists specifically to flag a duplicate institution identity as a data-quality problem. The harness asks a sharper question than "does the alert fire" – it asks what else in the codebase silently assumed that identity was unique.

5 The Governance Layer

Built on top of the published registry, the directorate console is organized into fourteen sections across three groups – network oversight (registry, districts, benchmarking, alerts, trends, union catalogue search), engagement (follow-ups, announcements, documents, inspections), and governance (reports, an audit log, and staff administration) – and introduces an access-tier model with three levels: full administrative access, a regional tier that can act on colleges but not manage other staff, and a read-only analyst tier.

BOOTSTRAP-SAFETY, NOT LEAST-PRIVILEGE

An account with no entry in the staff-tier collection resolves to the most privileged tier, not the least. This inverts the conventional default deliberately: every account that could sign in before the tier system existed must keep working after it ships, with no migration step, and a fail-closed default here would have locked out the one account the system most needs to keep working – the account doing the deploying. The same default is enforced twice, independently, in the security rules and in the client, so the client's UI and the server's enforcement can never silently disagree about who is allowed to do what.

Every state-changing action in the console – admitting or removing a college, enrolling or removing staff, publishing an announcement, scheduling an inspection – appends an entry to an audit log collection that permits create only, with update and delete unconditionally denied at the rules layer, including to the most privileged tier. An audit trail its own subjects can edit is not a trail; the append-only guarantee is enforced where it cannot be bypassed by a UI change.

6 Results: Three Defects at Scale

Run after the rules suite passed in full and the production build compiled cleanly across every route, the scale harness surfaced three defects, none of which any earlier layer of verification had caught. Table 2 summarizes them; the discussion below traces each to its root cause.

#	SYMPTOM	WHERE OBSERVED	ROOT CAUSE
D1	Rendering warning; rows silently duplicated or dropped	Three list views keyed on the institution's self-reported identity	That identity is not guaranteed unique
D2	A bounded search query consumed twice its intended budget on one institution and returned duplicate hits	Cross-tenant catalogue search	Same unenforced identity assumption
D3	A rejected account was returned to the login screen with no explanation shown	Authentication denial path	Two asynchronous auth-state callbacks racing; unrelated to D1/D2

Table 2. Defects recovered by the 300-institution scale harness. D1 and D2 share a root cause; D3 does not.

6.1 D1 and D2: one assumption, two failures

The registry's own alerting logic (Section 3.2) already treats a duplicate institution identity as a data-quality violation worth flagging to an administrator. What it does not do – because detecting a violation and defending against its consequences are different obligations – is prevent the rest of the codebase from silently assuming the identity is unique anyway. Three list views used that identity directly as a rendering key; when the harness's seeded duplicate reached them, two documents claiming the same identity produced a key collision, which under the client framework's reconciliation model risks silently duplicated or dropped rows rather than a visible crash. The cross-tenant search feature made the same assumption in a different form: it treated each registry entry as a distinct search target, so the duplicate caused it to query one tenant's data twice, spending two of its fixed forty-institution query budget on a single institution and returning every result twice.

The fix in both cases was the same: introduce a second identifier – the registry document's own storage identity, which the platform does guarantee unique – for anything that needs an identifier a hostile or merely confused publisher cannot forge, and reserve the institution's self-reported identity for what it was always meant for, addressing and linking to the college a user recognizes by name.

GENERALIZABLE FINDING

An alert that detects a uniqueness violation is not, by itself, a guarantee that the rest of the system tolerates the violation gracefully. Every other component that consumes the same collection and assumes the field is a valid key is exposed to the same failure the alert was built to report, until each one is checked independently. Detection and defense are separate engineering obligations, and building one does not retire the need for the other.

6.2 D3: a race in the denial path

Signing out a rejected account triggers the authentication client's own state-change notification, which runs a second, independent handler whose job is to clear session state when no user is signed in. That handler unconditionally cleared the rejection message the first handler had just set, before the interface had a chance to render it, because the sign-out call's internal notification completed before the handler that issued it returned. The account was silently returned to the login screen with no explanation – arguably a worse outcome for the person experiencing it than an incorrect password would have been, because a wrong-password message at least explains itself. The fix reorders the two operations so the explanatory state is set only after the sign-out call has fully settled, which is not a fix to any part of the duplicate-identity story; its recovery in the same testing pass and by the same methodology as D1 and D2 is itself part of the finding: a scale harness that drives real user flows end-to-end catches defect classes a rules-behavior suite, which asserts on data access rather than on interface behavior, structurally cannot.

7 Discussion

Five points generalize beyond this specific deployment.

Aggregation without access is achievable on a zero-compute backend, at the cost of trusting each tenant to report honestly about itself – a materially smaller trust requirement than trusting a shared backend to enforce isolation correctly, since a lying tenant can only misrepresent its own counts and cannot touch another tenant's data or approval status.

Bootstrap-safety and least-privilege are in tension, and a system introducing an access-tier model onto an existing user base should resolve that tension explicitly rather than by accident. Defaulting an unenrolled account to full access until it is deliberately narrowed is safer for this class of system than the conventional fail-closed default, specifically because the failure mode of fail-closed here – locking out the administrator who would fix it – has no recovery path once every account requires the very tier system that is broken.

Detection is not defense. Section 6.1's finding – that flagging a data-quality violation does not protect the rest of the codebase from it – almost certainly generalizes to any system that alerts on a uniqueness or integrity violation in data it does not have the authority to reject outright, which describes most federated or client-published data models.

Divergent trust models across clients hide defects asymmetrically. A rules regression invisible to an Admin-SDK client and fatal to a rules-governed one (Section 3.3) means the choice of which client is used to validate a change determines what that validation can see; testing strategy has to be platform-aware for exactly this reason.

Realistic irregularity finds what uniform fixtures cannot. All three defects in Section 6 required data that was irregular in a specific, deliberate way – not merely large. A scale test that seeds 300 identical, well-formed rows would have exercised none of them; it would have proven only that the system renders a large table, which was never in doubt.

8 Limitations

- **Single deployment context.** The architecture and its trade-offs are argued to generalize to similar public-sector, resource-constrained deployments; they are not validated against a second one.
- **Emulator-based scale testing approximates, but does not replicate, production conditions** – real network latency, concurrent writes from independent devices, and Android device/OS heterogeneity are all outside what a local emulator run can exercise.
- **No adversarial security review beyond behavioral rules testing** has been performed; the rules suite proves the boundary holds against the account classes and access patterns it enumerates, not against deliberate, creative attack.
- **The desktop client's Admin-SDK trust bypass (Section 3.3) remains an open architectural risk**, documented and reasoned about here but not resolved by this work.
- **The system's cost model is a static formula**, not a production-measured bill; validating it against real usage telemetry is future work, not something this paper reports as measured.

9 Conclusion

NEXLIB shows that a multi-tenant system with genuine cross-tenant oversight is achievable on a zero-compute, free-tier serverless backend, provided the aggregation is restructured as something each tenant publishes about itself rather than something a trusted server computes on the network's behalf. That restructuring produces its own trust model, its own governance requirements, and – as this paper's empirical half shows – its own failure class: assumptions about identity uniqueness that a data-quality alert can report but cannot, by itself, enforce. A verification strategy built only from a passing type-check, a clean build, and a rules-behavior suite would have shipped all three defects reported here. Finding them required treating "does it work" as a question about realistic, irregular data at scale, not a question about whether the code compiles.

References

- [1] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan. "Local-first software: you own your data, in spite of the cloud." *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!)*, 2019.
 - [2] E. A. Brewer. "Towards Robust Distributed Systems." Keynote, *ACM Symposium on Principles of Distributed Computing (PODC)*, 2000. Formalized in S. Gilbert and N. Lynch, "Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services," *ACM SIGACT News*, 2002.
 - [3] Google. "Cloud Firestore Security Rules." Firebase documentation, firebase.google.com/docs/firestore/security/get-started.
 - [4] R. Heeks. "Information Systems and Developing Countries: Failure, Success, and Local Improvisation." *The Information Society*, 18(2), 2002.
 - [5] OWASP Foundation. "OWASP Top 10." owasp.org/Top10/.
 - [6] NEXLIB project repository. Source, security rules, and rules-behavior and scale-test suites referenced throughout this paper. Private repository, Higher Education Department, Khyber Pakhtunkhwa deployment.
-

APPENDIX A

Directorate access-tier permission matrix

CAPABILITY	FULL ACCESS	REGIONAL	ANALYST
Read every section	Yes	Yes	Yes
Admit / remove a college	Yes	Yes	No
Write follow-ups, announcements, documents, inspections, notes	Yes	Yes	No
Enroll or remove staff records	Yes	No	No
Default when unenrolled	Full access — see Section 5, bootstrap-safety note		

APPENDIX B

Verification summary at time of writing

CHECK	RESULT
Behavioral rules assertions	75 / 75 passing
Production build	Compiles cleanly; all 17 routes generated
Type-check	Clean
Scale-harness console errors, pre-fix	3 distinct errors across 3 pages, all traced to D1
Scale-harness console errors, post-fix	0, across all 14 sections and the institution detail view
Write paths exercised live at n=300 and confirmed in the audit log	10 of 10 (admit/remove a college, follow-up create/resolve, announcement publish/retract, document add/remove, inspection schedule/complete, staff enroll/remove, note save, trend snapshot capture)